Atividade 2 HTML + CSS

As 10 Heurísticas de Nielsen

Isabela Alvim - 22610970

Código

HTML:

```
<!DOCTYPE html> <html lang="en">

<head>

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Atividade 02 - Heurísticas Nielsen</title>

    <link rel="stylesheet" href="atv2.css">

</head>

<body>

    <h1>As 10 Heurísticas de Nielsen</h1>

<section id="Introdução"> <h2>Introdução</h2> </section>
<p>
    <ul>
        As 10 Heurísticas de Nielsen são princípios utilizados para avaliar a usabilidade e facilidade de uso de sistemas, sites e aplicativos.
        <br>O objetivo não é fazer o software funcionar por trás dos panos, mas sim garantir que a tela e o fluxo façam sentido para a cabeça de quem está usando.
        <br>É o ponto de encontro entre a psicologia humana e o código.
        <br>Criadas pelo especialista Jakob Nielsen, elas ajudam a identificar problemas na interface e melhorar a experiência do usuário.
        <br>De forma simples, funcionam como um guia para verificar se um sistema é fácil de entender, utilizar e aprender.
      </ul>


<br>
 <section id="inicio"> <h2>Um pouco sobre as 10 Heurísticas e o que cada uma significa de modo geral</h2> </section>

<p>
```

```
    <h3>01. Visibilidade do status do sistema</h3>
    <ul>
    O sistema deve manter o usuário informado sobre o que está acontecendo. As respostas do sistema devem aparecer em um tempo adequado.
    Exemplo: ao fazer um download, mostrar uma barra de progresso ou a mensagem “Baixando...”.
    <br><br>
    Em resumo: o usuário nunca deve ficar se perguntando se uma ação foi realizada ou se o sistema está funcionando.
      </ul>
    <br>

    <h3>02. Correspondência entre o sistema e o mundo real</h3>
    <ul>
    O sistema deve utilizar uma linguagem que seja familiar ao usuário.
    Deve evitar termos excessivamente técnicos ou relacionados à programação.
    As informações devem seguir uma organização que faça sentido no mundo real.
    Exemplo: utilizar um ícone de lixeira para representar “Excluir”.
    <br><br>
    Em resumo: o sistema deve falar a “língua” do usuário, e não a linguagem do programador.
      </ul>
    <br>

    <h3>03. Controle e liberdade do usuário</h3>
    <ul>
    O usuário pode cometer erros e precisa ter uma maneira simples de voltar atrás.
    É importante oferecer opções como cancelar, voltar, desfazer e refazer.
    Exemplo: depois de excluir um arquivo, oferecer a opção “Desfazer”.
    <br><br>
    Em resumo: o usuário deve sentir que está no controle da aplicação, e não preso a uma ação.
    </ul>
    <br>

    <h3>04. Consistência e padrões</h3>
    <ul>
    Elementos semelhantes devem funcionar e ser apresentados de maneira semelhante em todo o sistema.
    Os termos, botões, cores e comportamentos devem seguir padrões conhecidos.
    Exemplo: se o botão “Salvar” é representado por um determinado ícone em uma tela, ele não deve representar outra função em outra tela.
        <br><br>
    Em resumo: manter padrões reduz a necessidade de o usuário aprender novamente como cada parte do sistema funciona.
```

```
    </ul>
    <br>

    <h3>05. Prevenção de erros</h3>
    <ul>
    É melhor evitar que o erro aconteça do que simplesmente mostrar uma mensagem depois que ele ocorreu.
    O sistema deve identificar situações que podem causar problemas e orientar o usuário.
    Exemplo: antes de excluir definitivamente uma conta, apresentar uma confirmação.
    <br><br>
    Em resumo: um bom sistema não apenas sabe lidar com erros, mas também tenta impedir que eles aconteçam.
    </ul>
    <br>

    <h3>06. Reconhecimento em vez de memorização</h3>
    <ul>
    O sistema deve facilitar o reconhecimento de informações, evitando exigir que o usuário memorize comandos ou informações.
    Opções e instruções importantes devem estar visíveis ou facilmente acessíveis.
    Exemplo: menus que mostram as opções disponíveis em vez de exigir que o usuário memorize comandos.
    <br><br>
    Em resumo: é mais fácil reconhecer uma informação apresentada na tela do que lembrar dela sozinho.
    </ul>
    <br>

    <h3>07. Flexibilidade e eficiência de uso</h3>
    <ul>
    O sistema deve funcionar bem tanto para usuários iniciantes quanto para usuários experientes.
    Usuários avançados podem receber atalhos e recursos que tornam tarefas frequentes mais rápidas.
    Exemplo: utilizar Ctrl + C e Ctrl + V em vez de acessar o menu toda vez que deseja copiar e colar.
    <br><br>
    Em resumo: o sistema deve ser simples para quem está começando, mas também permitir mais agilidade para quem já possui experiência.
    </ul>
    <br>

    <h3>08. Estética e design minimalista</h3>
    <ul>
```

```
    A interface deve apresentar somente as informações realmente relevantes para a tarefa.
    Excesso de textos, botões, imagens ou informações pode dificultar a compreensão.
    Exemplo: uma tela de login normalmente precisa apenas dos campos necessários para entrar na conta, sem várias informações que distraiam o usuário.
    <br><br>
    Em resumo: quanto mais informações desnecessárias aparecem, mais difícil fica encontrar aquilo que realmente importa.
    </ul>
    <br>

    <h3>09. Ajudar o usuário a reconhecer, diagnosticar e corrigir erros</h3>
    <ul>
    Quando um erro acontece, a mensagem deve explicar o que aconteceu e, quando possível, como resolver.
    As mensagens devem utilizar uma linguagem simples, evitando códigos técnicos que o usuário não entende.
    Exemplo ruim: “Erro 0x80070005”.
    Exemplo melhor: “Não foi possível salvar o arquivo. Verifique se você possui permissão para acessar esta pasta.”
    <br><br>
    Em resumo: uma boa mensagem de erro não apenas informa o problema, mas ajuda o usuário a encontrar uma solução.
    </ul>
    <br>

    <h3>10. Ajuda e documentação</h3>
    <ul>
    O ideal é que o sistema seja fácil de utilizar sem precisar consultar um manual.
    Porém, quando uma função é mais complexa, é importante disponibilizar ajuda, tutoriais ou documentação.
    Essas informações devem ser fáceis de encontrar e explicar de forma objetiva como realizar determinada tarefa.
    Exemplo: um botão “?” que abre uma explicação sobre uma função específica.
    <br><br>
    Em resumo: quando o usuário precisar de ajuda, ela deve estar disponível e ser fácil de entender.
    </ul>
    <br><br>


<section id="Conclusão"> <h2>Conclusão</h2> </section>
<p>
    <ul>
```

```
        As 10 Heurísticas de Nielsen são importantes na Engenharia de Software e na área de UX (User Experience) porque ajudam a avaliar se um sistema é fácil de aprender, compreender e utilizar.
        <br>Na prática, elas funcionam como uma espécie de checklist de usabilidade: ao analisar uma aplicação, podemos verificar se o usuário recebe feedback, consegue desfazer ações, entende os termos utilizados, evita erros e encontra ajuda quando necessário.
        <br>De forma simples, funcionam como um guia para verificar se um sistema é fácil de entender, utilizar e aprender.
      </ul>


<br><br><br>
    <h2>Referencias:</h2>
    <ul>
        <li>NIELSEN NORMAN GROUP. 10 Usability Heuristics for User Interface Design. Jakob Nielsen.</li>
        <li>NIELSEN NORMAN GROUP. Jakob’s Ten Usability Heuristics - Heuristic Summary.</li>
        <li>NIELSEN, Jakob. 10 Heuristics for User Interface Design.</li>
        <li>MIT - Massachusetts Institute of Technology. Heuristic Evaluation and User Centered Design.</li>
    </ul>


</body>

</html>
```

CSS:

```
body {
    background-color: #ffe6f0;
    font-family: Arial, sans-serif;
    color: #4a2c3a;
    margin: 0;
}

h1 {
    background-color: #d94f8a;
    color: white;
    text-align: center;
    padding: 30px;
    margin: 0;
}
```

```css
h2 {
    color: #c13b78;
    border-bottom: 2px solid #e89ab8;
    padding-bottom: 8px;
}

h3 {
    color: #b83268;
}

p {
    font-size: 16px;
    line-height: 1.6;
}

ul {
    line-height: 1.7;
}

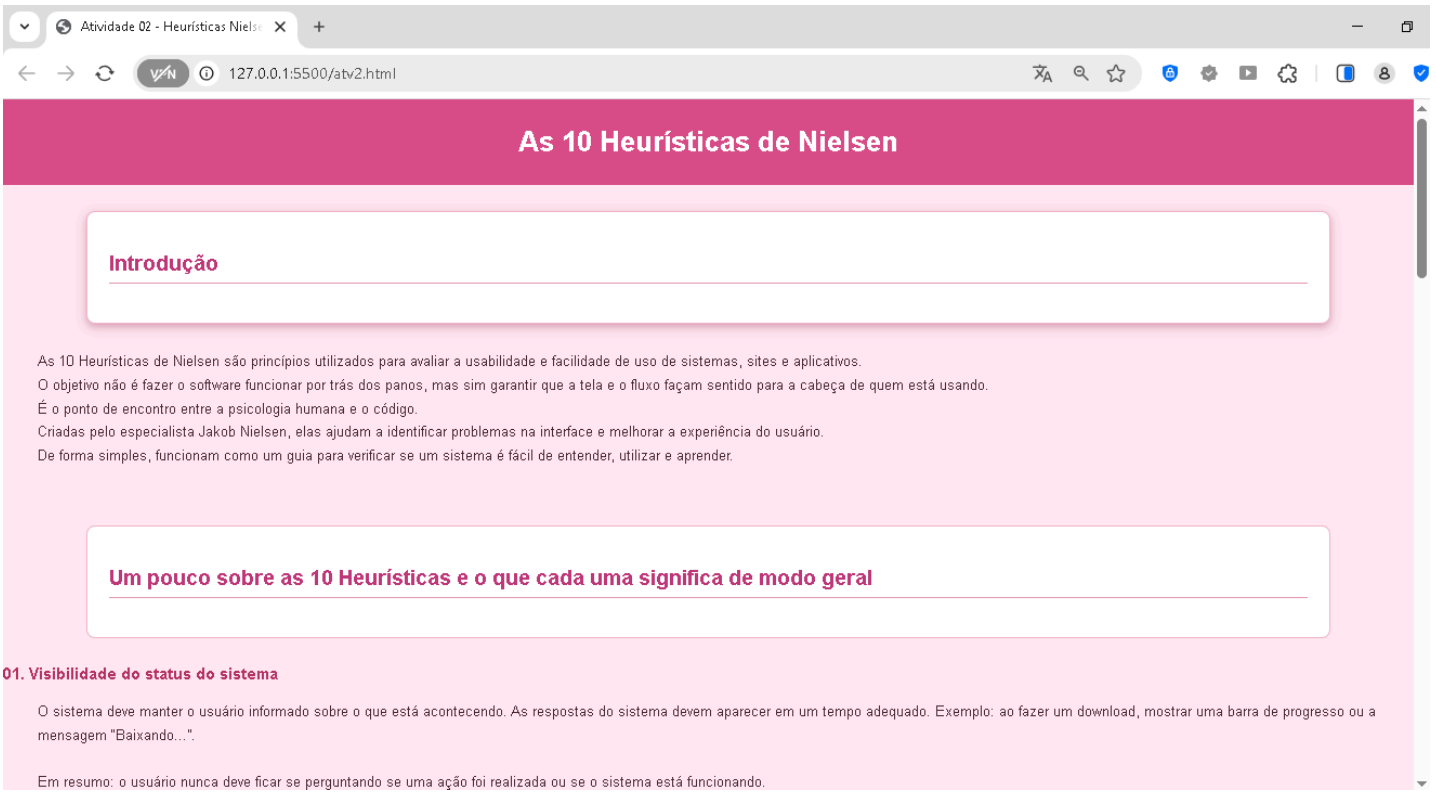
section {
    background-color: white;
    width: 85%;
    margin: 30px auto;
    padding: 25px;
    border-radius: 10px;
    border: 1px solid #f0b6ca;
}

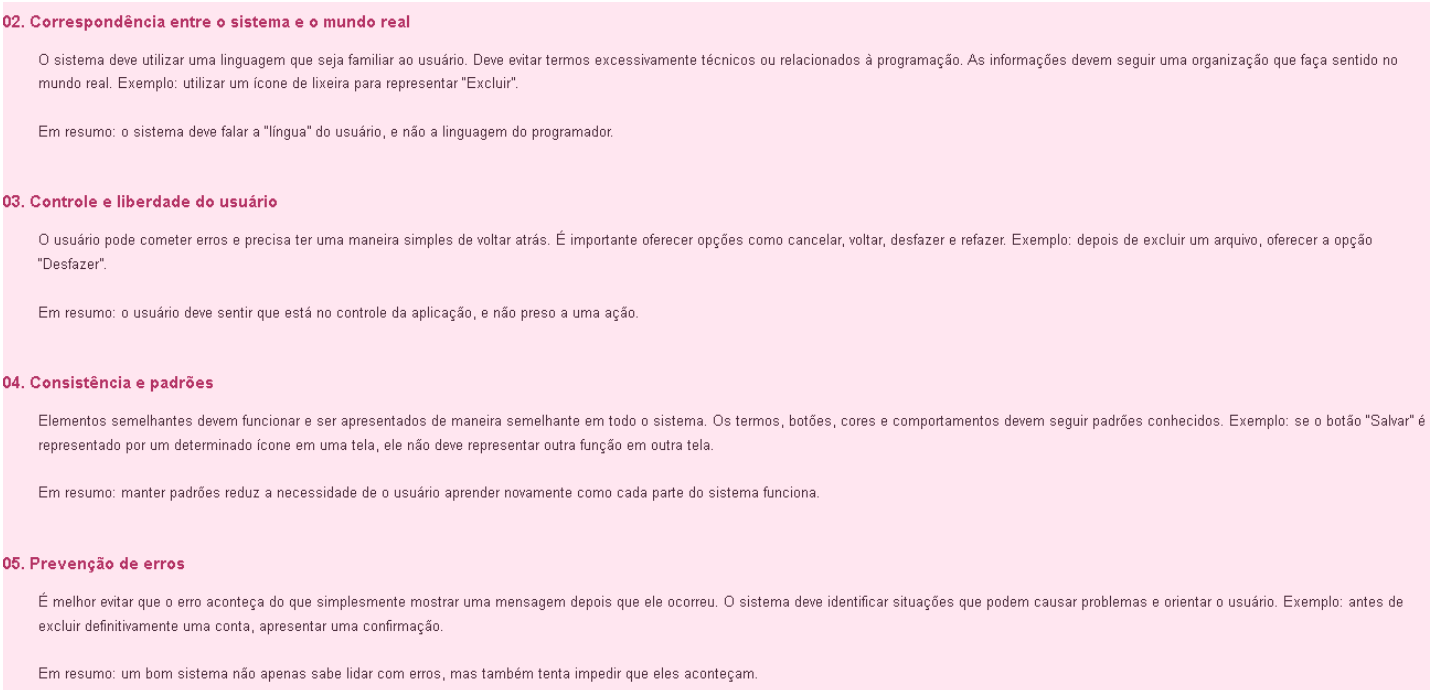
section:hover {
    box-shadow: 0 4px 10px #d9a0b5;
}

.icone {
    margin-right: 8px;
    font-size: 24px;
}

li {
    margin-bottom: 10px;
}
```

Prints da página





06. Reconhecimento em vez de memorização

O sistema deve facilitar o reconhecimento de informações, evitando exigir que o usuário memorize comandos ou informações. Opções e instruções importantes devem estar visíveis ou facilmente acessíveis. Exemplo: menus que mostram as opções disponíveis em vez de exigir que o usuário memorize comandos.

Em resumo: é mais fácil reconhecer uma informação apresentada na tela do que lembrar dela sozinho.

07. Flexibilidade e eficiência de uso

O sistema deve funcionar bem tanto para usuários iniciantes quanto para usuários experientes. Usuários avançados podem receber atalhos e recursos que tornam tarefas frequentes mais rápidas. Exemplo: utilizar Ctrl + C e Ctrl + V em vez de acessar o menu toda vez que deseja copiar e colar.

Em resumo: o sistema deve ser simples para quem está começando, mas também permitir mais agilidade para quem já possui experiência.

08. Estética e design minimalista

A interface deve apresentar somente as informações realmente relevantes para a tarefa. Excesso de textos, botões, imagens ou informações pode dificultar a compreensão. Exemplo: uma tela de login normalmente precisa apenas dos campos necessários para entrar na conta, sem várias informações que distraiam o usuário.

Em resumo: quanto mais informações desnecessárias aparecem, mais difícil fica encontrar aquilo que realmente importa.

09. Ajudar o usuário a reconhecer, diagnosticar e corrigir erros

Quando um erro acontece, a mensagem deve explicar o que aconteceu e, quando possível, como resolver. As mensagens devem utilizar uma linguagem simples, evitando códigos técnicos que o usuário não entende. Exemplo ruim: "Erro 0x80070005". Exemplo melhor: "Não foi possível salvar o arquivo. Verifique se você possui permissão para acessar esta pasta."

Em resumo: uma boa mensagem de erro não apenas informa o problema, mas ajuda o usuário a encontrar uma solução.

10. Ajuda e documentação

O ideal é que o sistema seja fácil de utilizar sem precisar consultar um manual. Porém, quando uma função é mais complexa, é importante disponibilizar ajuda, tutoriais ou documentação. Essas informações devem ser fáceis de encontrar e explicar de forma objetiva como realizar determinada tarefa. Exemplo: um botão "?" que abre uma explicação sobre uma função específica.

Em resumo: quando o usuário precisar de ajuda, ela deve estar disponível e ser fácil de entender.

Conclusão

As 10 Heurísticas de Nielsen são importantes na Engenharia de Software e na área de UX (User Experience) porque ajudam a avaliar se um sistema é fácil de aprender, compreender e utilizar.
Na prática, elas funcionam como uma espécie de checklist de usabilidade: ao analisar uma aplicação, podemos verificar se o usuário recebe feedback, consegue desfazer ações, entende os termos utilizados, evita erros e encontra ajuda quando necessário.
De forma simples, funcionam como um guia para verificar se um sistema é fácil de entender, utilizar e aprender.

Referencias:

- NIELSEN NORMAN GROUP. 10 Usability Heuristics for User Interface Design. Jakob Nielsen.
- NIELSEN NORMAN GROUP. Jakob's Ten Usability Heuristics - Heuristic Summary.
- NIELSEN, Jakob. 10 Heuristics for User Interface Design.
- MIT - Massachusetts Institute of Technology. Heuristic Evaluation and User Centered Design.